

STOQUIFY

Stoquify Internal Activation Gate Remediation and Controlled-Pilot Readiness Execution

EXECUTION PROMPT

Repository remediation program for controlled internal-pilot readiness.

Prepared 23 July 2026

No production activation is authorized by this document.

Act as a principal multidisciplinary implementation team comprising:

- Enterprise system architect
- Application security architect
- DevSecOps and platform engineer
- Site reliability and observability engineer
- Prisma and PostgreSQL specialist
- Playwright and release-certification lead
- AI governance specialist
- Product rollout and support operations lead
- Financial systems control and audit specialist

Source Report

Use this report as the authoritative starting point:

`what-next/agents-runtime/STOQUIFY_INTERNAL_ACTIVATION_GATE_UNBLOCKING_ANALYSIS_AND_EXECUTION_PLAN_2026-07-23.md`

Inspect the current Stoquify repository, database schema, migrations, services, protected actions, deployment configuration, CI workflows, tests, and existing operational infrastructure before making changes.

Do not rely exclusively on the report. Verify every stated condition against the current codebase and deployed evidence available in the workspace.

Primary Objective

Practically resolve every repository-addressable problem currently blocking Stoquify Agent Runtime Phase 2A from proceeding to a controlled internal pilot.

The target state is:

READY FOR CONTROLLED INTERNAL PILOT

Do not activate production or expose the agent to real users during this execution. Build, test, and certify the controls required for a separately authorized activation.

The five blocked gates are:

1. Recorded product and security approval
2. Named rollout, rollback, support, pilot, security-incident, and backup owners
3. Deployed reconciliation schedule and secure service authentication
4. Production-capable alert transport and accountable ownership
5. Enabled-pilot Playwright certification

Execution Principles

- Reuse Stoquify's existing agent runtime, protected-action boundary, RBAC system, audit log, Workflow Assurance incident spine, and release-gate conventions.
- Add the least code that creates enforceable, auditable controls.
- Do not introduce another agent framework or duplicate Stoquify's incident infrastructure.

- Keep the Command Agent deterministic, provider-free, evidence-backed, and read-only.
- Preserve tenant isolation, idempotency, fail-closed behavior, auditability, and rollback.
- Do not fabricate approver names, owner identities, pilot tenants, contact details, deployment providers, secrets, or production evidence.
- Implement technical structures for missing human decisions, then generate a precise completion register for authorized people to fill.
- Do not expose or print secret values.
- Do not perform direct production database mutations.
- Do not permit direct Prisma writes from agents.
- Do not permit direct ledger posting, statutory filing, payment execution, inventory adjustment, payroll action, approval, or permission changes.
- Stop if a Critical or High security invariant fails.

Required Work

Phase 1: Revalidate the Current Blocked State

Confirm and document:

- Current agent, skill, and tool definition statuses
- Current rollout and kill-switch behavior
- Whether DRAFT/SHADOW records exist
- Whether any activation package or approval record already exists
- Whether the provisioning script can still activate definitions directly
- Reconciler functionality and deployment state
- Scheduler and secret configuration
- Logger and alert-delivery behavior
- Existing Workflow Assurance infrastructure
- Current Playwright coverage
- Current CI release gates
- Production or pilot deployment evidence available in the repository

Distinguish clearly between:

- Implemented
- Configured
- Deployed
- Tested
- Certified
- Formally approved
- Activated

Phase 2: Build the Release-Control Data Model

Implement an auditable release-control schema for:

- Activation packages
- Product and security approvals
- Operational owner assignments
- Pilot certification
- State transitions and audit evidence

Support this fail-closed lifecycle:

DRAFT -> REVIEWED -> APPROVED -> PROVISIONED_INACTIVE -> PILOT_CERTIFIED -> ACTIVE_INTERNAL -> SUSPENDED -> RETIRED

The activation package must bind:

- Agent key
- Release version
- Git commit SHA
- Complete manifest and manifest hash
- Agent, skill, tool, schema, and prompt versions
- Model-provider policy, which must remain none for Phase 2A
- Target environment
- Pilot organization
- Approved roles
- Activation window
- Known and accepted residual risks
- Product and security approvals
- Operational owners
- Certification evidence

Add appropriate Prisma relations, indexes, uniqueness constraints, immutability rules, expiry rules, and audit records.

Run migration safety checks and PostgreSQL tests.

Phase 3: Implement Governed State Transitions

Create one canonical release-control service for:

- Preparing a release package
- Freezing the manifest for review
- Recording product approval
- Recording security approval
- Recording rejection or revocation
- Assigning and acknowledging owners
- Provisioning inactive or shadow definitions
- Recording pilot certification

- Activating a controlled internal release
- Suspending a release
- Retiring a release

Add protected permissions such as:

- `agent.release.view`
- `agent.release.prepare`
- `agent.release.approve.product`
- `agent.release.approve.security`
- `agent.release.activate`
- `agent.release.suspend`

Enforce:

- Fresh authentication for approval and activation
- Separation of duties
- Optimistic concurrency
- Idempotency
- Exact manifest matching
- Approval expiry and revocation
- Valid activation windows
- Complete ownership
- Current certification
- Reconciliation health
- Alert-transport health
- Append-only audit evidence

The agent runtime must never possess release-control permissions.

Phase 4: Eliminate Direct Activation Bypass

Refactor the Phase 2A provisioning script so it cannot directly mark definitions ACTIVE.

The script may:

- Produce a dry-run plan
- Generate a release manifest and hash
- Provision DRAFT records
- Invoke the canonical guarded transition service with an approved release-package ID

The script must not:

- Set ACTIVE based only on `--activate`
- Bypass approvals
- Bypass owner validation
- Bypass certification
- Bypass reconciliation or alert-health checks

- Mutate production definitions without an auditable release transition

Add a static release gate that fails if scripts or unauthorized services directly activate agent definitions.

Phase 5: Implement Approval and Ownership Workflows

Build the technical workflow for recording:

- Product approver
- Security approver
- Rollout owner
- Rollback owner
- Support owner
- Pilot owner
- Security-incident owner
- On-call backup

Each owner assignment must include:

- Directory user identity
- Primary and backup responsibility
- Acknowledgement timestamp
- Validity period
- Escalation-route reference
- Coverage window
- Runbook version accepted

Generate a RACI record and human-completion register.

Do not mark this gate complete until real authorized identities and approvals have been supplied.

Phase 6: Expand and Deploy Reconciliation

Extend reconciliation beyond abandoned runs.

The scheduled reconciliation must compare:

- Approved release manifest versus deployed build
- Code manifest versus PostgreSQL definitions
- Definition status and rollout mode
- Agent and skill versions
- Prompt and schema hashes
- Allowed skills and tools
- Tool permission and policy assignments
- Pilot organization and role scope
- Product and security approval validity
- Owner validity and acknowledgement
- Playwright certification validity

- Active release uniqueness
- Unauthorized or duplicate records
- Open Critical incidents
- Reconciliation heartbeat
- Alert-delivery health

Reuse the Workflow Assurance engine for check runs, findings, incidents, events, and evidence.

Add an AGENT_RUNTIME workflow category if technically appropriate.

Implement:

- Five-minute scheduling
- Idempotent scheduled windows
- Bounded execution
- Retry with backoff and jitter
- Persistent heartbeat
- Missing-heartbeat detection
- Safe failure codes
- Critical-drift suspension
- Non-destructive reconciliation
- Alert integration

Prefer workload identity or OIDC. If bearer authentication is required, use a managed secret of adequate entropy and provide rotation support without revealing its value.

If the deployment provider is not discoverable, implement the provider-neutral job boundary and generate the exact deployment adapter still required. Do not claim the scheduler is deployed until deployment evidence exists.

Phase 7: Complete Alerting and Incident Response

Reuse and complete Stoquify's Workflow Assurance alert-delivery infrastructure.

Implement a shared delivery worker that:

- Claims pending deliveries safely
- Supports bounded retries
- Deduplicates alerts
- Uses safe, redacted payloads
- Records delivered and failed outcomes
- Dead-letters exhausted deliveries
- Supports primary and secondary transports
- Records external incident references
- Detects its own delivery failure

Create agent-runtime alert rules for:

- Unauthorized activation
- Manifest or tenant-scope drift

- Approval expiry or revocation
- Missing reconciliation heartbeat
- Cross-tenant authorization anomalies
- Repeated policy denials
- Timeouts and repeated execution failures
- Abandoned runs
- Unsafe feedback
- Stale-output rate
- Audit-write failures
- Alert-delivery backlog

Add dashboards for runtime health, incidents, reconciliation, delivery status, and pilot service objectives.

Do not mark this gate complete until a production-like test alert reaches a named owner, is acknowledged, and demonstrates escalation.

Phase 8: Build the Enabled-Pilot Test Environment

Create an isolated E2E environment containing:

- Approved pilot organization
- Non-pilot organization
- Approved pilot user and role
- Denied-role user
- Non-pilot user
- Fresh, stale, partial, empty, and blocked evidence fixtures
- Test approval records
- Test owner assignments
- Exact release manifest
- Active definitions provisioned through the guarded transition service

Do not activate real production organizations.

Keep fixture creation and cleanup tenant-scoped and repeatable.

Phase 9: Implement Enabled-Pilot Playwright Certification

Add required Playwright projects for:

- Enabled pilot desktop
- Enabled pilot mobile
- Unauthorized tenant
- Unauthorized role
- Missing permission
- Module denial
- DRAFT definition
- Manifest mismatch

- Evidence navigation
- Stale, partial, empty, timeout, retry, and replay behavior
- Feedback persistence
- Audit evidence
- Package suspension
- Kill switch
- Rollback
- Keyboard operation
- Accessibility
- Responsive layout

The enabled tests must use real authentication, PostgreSQL, RBAC, tenant resolution, definitions, protected actions, evidence persistence, and audit records.

Add database assertions proving:

- No prohibited business table was mutated
- No direct ledger, filing, payment, stock, payroll, approval, or permission write

occurred

- Cross-tenant records were not exposed
- Runs and evidence remained correctly scoped

Bind certification to:

- Git commit SHA
- Manifest hash
- Test-suite version
- CI run ID
- Report hash
- Certification expiry

Phase 10: Integrate Mandatory CI Release Gates

Add required CI jobs for:

- Prisma validation
- Migration safety
- Agent static gates
- Direct-activation bypass gate
- Focused unit and integration tests
- PostgreSQL smoke tests
- Reconciliation drift tests
- Alert-delivery tests
- Enabled-pilot Playwright certification
- Kill-switch and rollback tests

- Evidence artifact validation

Retain failure traces, screenshots, videos, reports, and machine-readable summaries without including secrets or sensitive payloads.

Phase 11: Prepare Controlled Activation Evidence

Generate a complete activation package containing:

- Exact release identity
- Manifest and hash
- Migration evidence
- Test evidence
- Playwright certification
- Product and security approval status
- Owner status
- Reconciliation heartbeat
- Alert-transport health
- Open incident status
- Activation and rollback runbooks
- Go/no-go checklist

Do not activate the agent as part of this task unless separately and explicitly authorized after every gate passes.

Required Verification

Run and report, where applicable:

- Prisma schema validation
- Migration safety checks
- Controlled migration deployment
- PostgreSQL persistence and constraint tests
- TypeScript checks
- Focused ESLint
- Agent runtime static gates
- Direct-activation bypass gate
- Prohibited-action gate
- Unit and integration suites
- Reconciliation tests
- Alert-delivery tests
- Enabled-pilot Playwright matrix
- Kill-switch and rollback tests
- CI evidence checks
- `git diff --check`

Resolve failures caused by the implementation. Do not hide or silently waive failures.

Required Deliverables

Produce:

1. Current-state revalidation report
2. Files and components changed
3. Release-control architecture
4. Prisma schema and migration summary
5. Guarded state-transition implementation
6. Direct-activation bypass remediation
7. Approval and ownership workflow
8. Reconciliation architecture and deployment evidence
9. Secret-management and rotation design
10. Alert-delivery implementation and test evidence
11. Dashboard and alert-rule definitions
12. Enabled-pilot fixture and Playwright design
13. CI release-gate integration
14. Controlled activation runbook
15. Rollback and emergency-disable runbook
16. Human-decision completion register
17. Final gate matrix
18. Exact remaining blockers
19. Next authorized action

Completion Rules

A gate may be marked PASSED only when implementation, deployment or environment configuration, tests, ownership, and evidence all exist as required.

Use these final statuses:

- BLOCKED
- READY FOR REMEDIATION VERIFICATION
- READY FOR CONTROLLED INTERNAL PILOT
- APPROVED FOR INTERNAL ACTIVATION

Do not use APPROVED FOR INTERNAL ACTIVATION unless every technical and human gate is complete and separately authorized.

If human identities, production deployment access, or external alert credentials are unavailable:

- Complete every repository-addressable implementation.
- Generate exact configuration and evidence templates.
- Mark the affected gate as blocked by external input.

- State precisely who must provide what.
- Do not fabricate completion.

Final Decision

Conclude with:

- Gates passed
- Gates still blocked
- Verification results
- Human actions still required
- Deployment actions still required
- Whether Stoquify is ready for a controlled internal pilot
- Whether it may advance beyond Phase 2A
- The next technically executable and authorized action